

Actividad 09: Road to Hall Of Fame

Laboratorio: File path traversal, traversal sequences stripped non-recursively

Asignatura: Seguridad Informática

Realizado por el alumno: Manuel Darío Trejo Salazar

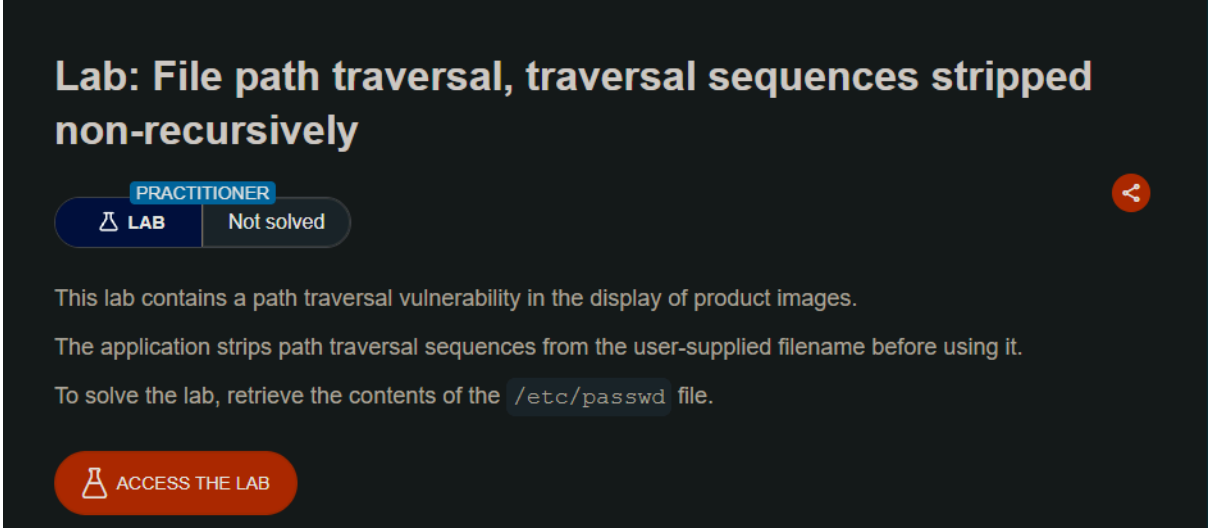
Herramienta utilizada: Burp Suite Community Edition

1.- Nombre del laboratorio

File path traversal, traversal sequences stripped non-recursively

URL del laboratorio:

<https://portswigger.net/web-security/file-path-traversal/lab-sequences-stripped-non-recursively>



The screenshot shows the PortSwigger lab interface. At the top, the title 'Lab: File path traversal, traversal sequences stripped non-recursively' is displayed in a large, bold font. Below the title, there is a 'PRACTITIONER' badge and a 'LAB' button with a flag icon. To the right of the 'LAB' button is a 'Not solved' status indicator. A red share icon is located in the top right corner. The main text describes the lab: 'This lab contains a path traversal vulnerability in the display of product images. The application strips path traversal sequences from the user-supplied filename before using it. To solve the lab, retrieve the contents of the `/etc/passwd` file.' At the bottom, there is a red button with a flag icon and the text 'ACCESS THE LAB'.

2.- Nivel Practitioner (Intermedio).

Este laboratorio presenta un filtro de seguridad que intenta sanitizar la entrada eliminando las secuencias de recorrido (../). El atacante debe evadirlo aprovechando que la eliminación no se realiza de forma recursiva.

3.- Tipo de explotación

File Path Traversal (también conocido como Directory Traversal o recorrido de directorios), evadiendo filtros de sanitización (Filter Bypass).

La vulnerabilidad ocurre cuando la aplicación implementa un filtro de seguridad defectuoso. El desarrollador utiliza una función de reemplazo de cadenas (como `str.replace("../", "")`) para eliminar las secuencias peligrosas de la entrada del usuario, pero solo lo ejecuta una vez por coincidencia (de forma secuencial y no recursiva).

Al no aplicar el filtro repetidamente hasta que no queden más secuencias, un atacante puede ofuscar el payload anidando caracteres (por ejemplo, `....//`).

Cuando la aplicación procesa esta cadena, elimina el `../` interior, y los caracteres restantes se unen para formar un nuevo `../` válido que el sistema de archivos interpretará sin problemas.

Consecuencias: Las mismas que un Path Traversal clásico (lectura arbitraria de archivos sensibles, código fuente y credenciales), demostrando que las listas negras y las funciones de reemplazo simples son ineficaces para sanitizar rutas.

4. Objetivo del laboratorio

Demostrar la explotación de una vulnerabilidad de File Path Traversal evadiendo un filtro de sanitización no recursivo. Esto se logra inyectando secuencias de recorrido anidadas (`....//`) a través del parámetro `filename` de la petición HTTP que carga las imágenes de los productos, con el fin de generar secuencias válidas en el backend, leer el contenido del archivo `/etc/passwd` del servidor y obtener la confirmación de "Lab solved" en la Web Security Academy.

5. Explicación técnica de la vulnerabilidad

La aplicación web procesa la carga de imágenes mediante la siguiente petición:
`GET /image?filename=71.jpg HTTP/2`

Para protegerse de ataques de recorrido de rutas, el backend incluye una línea de código que busca activamente la subcadena `../` (y posiblemente `..\`) y la reemplaza por un string vacío (`""`).

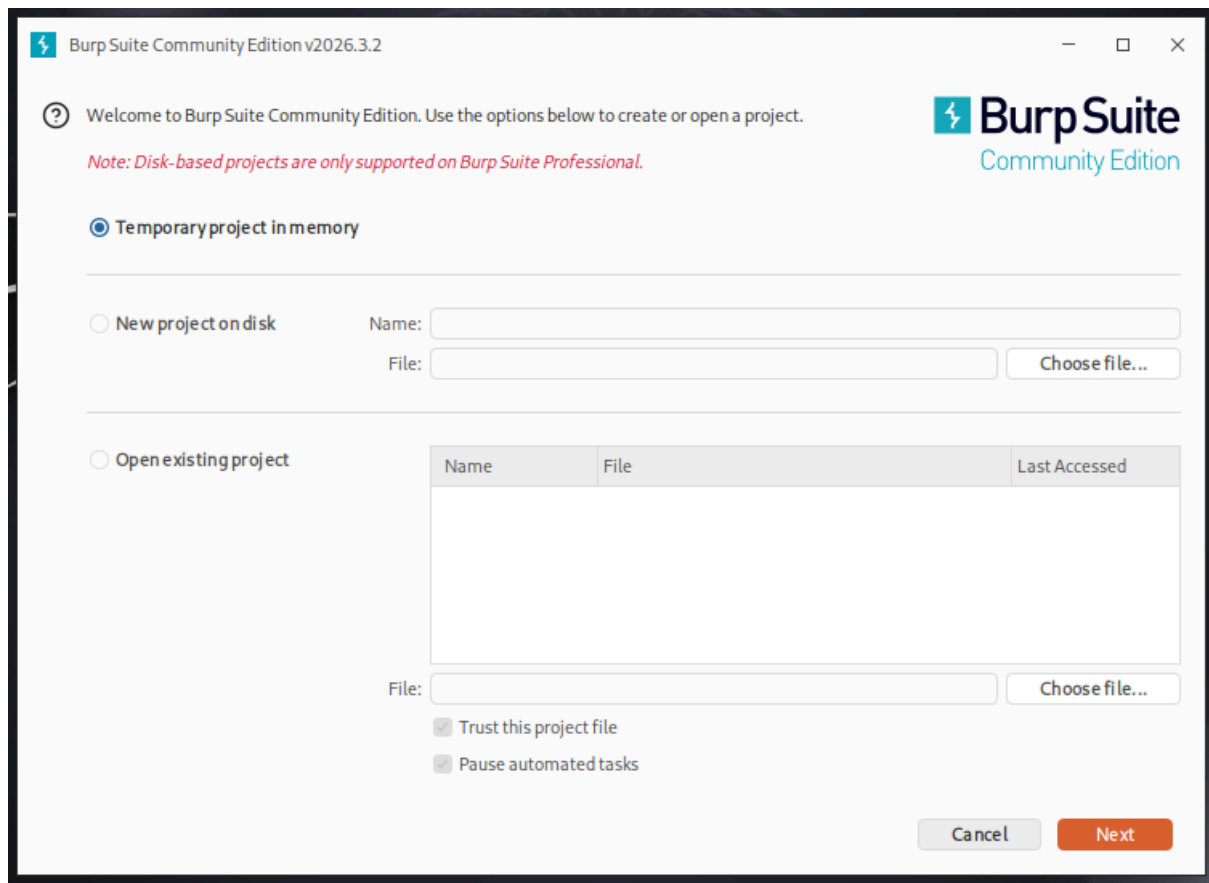
El error arquitectónico radica en que este mecanismo no es recursivo. Si la entrada del atacante es//, el filtro actúa de la siguiente manera:

- Analiza el string: .. + ../ + /
- Identifica la secuencia prohibida en el medio: ../
- La elimina.
- Concatena lo que queda: los primeros .. con la última /.
- El resultado final enviado al sistema de archivos es: ../

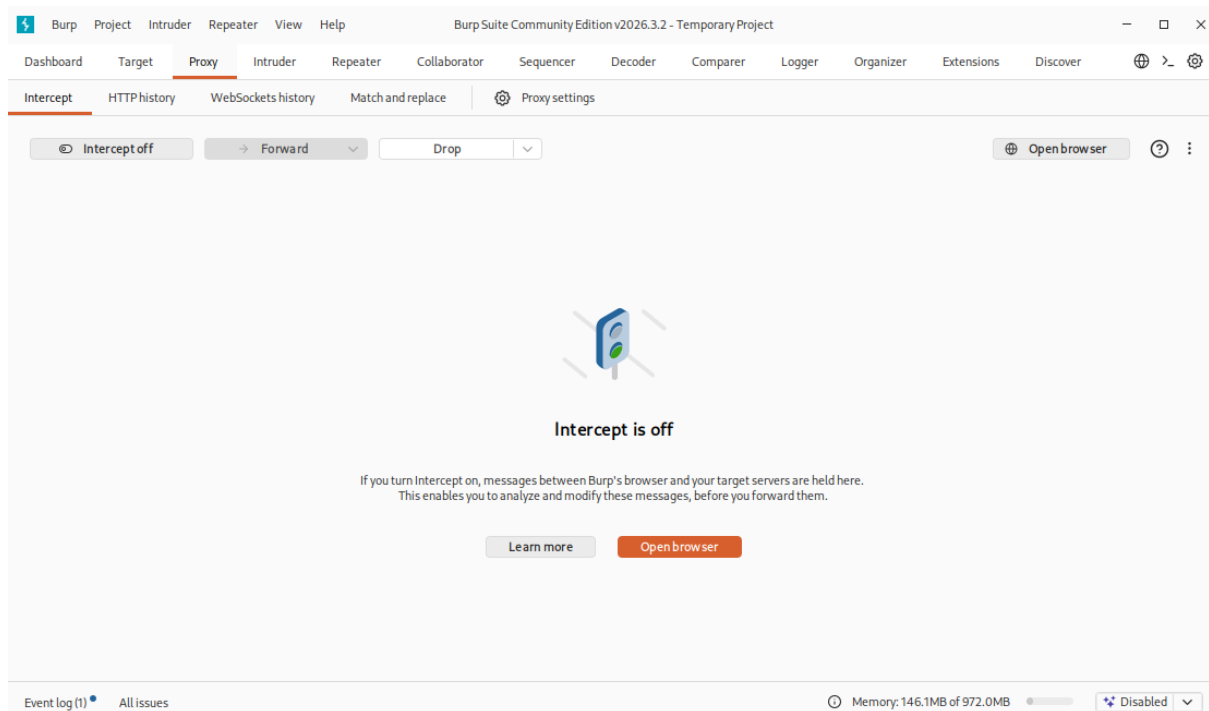
Dado que el filtro ya terminó su ciclo de ejecución, esta nueva secuencia resultante nunca es evaluada ni eliminada, permitiendo al atacante retroceder directorios en el sistema de archivos del servidor hasta llegar a la raíz.

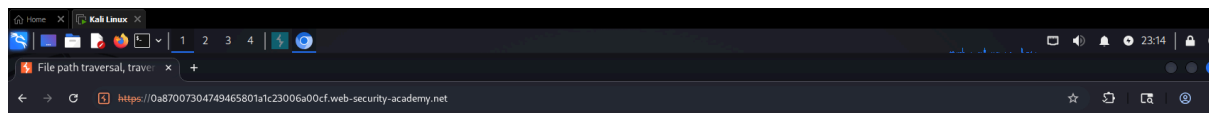
6. Procedimiento paso a paso

Se inicia Burp Suite Community Edition desde la máquina virtual, seleccionando en las configuraciones iniciales, Temporary project in memory y posteriormente Use Burp defaults.



Con Burp abierto en la pestaña Proxy > Intercept, se abre el navegador integrado y se accede a la URL del laboratorio proporcionada por PortSwigger.



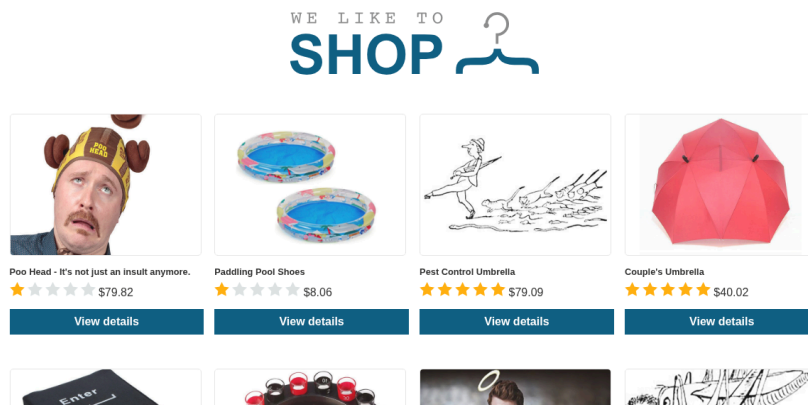


Web Security Academy

File path traversal, traversal sequences stripped non-recursively
[Back to lab description >>](#)

LAB Not solved

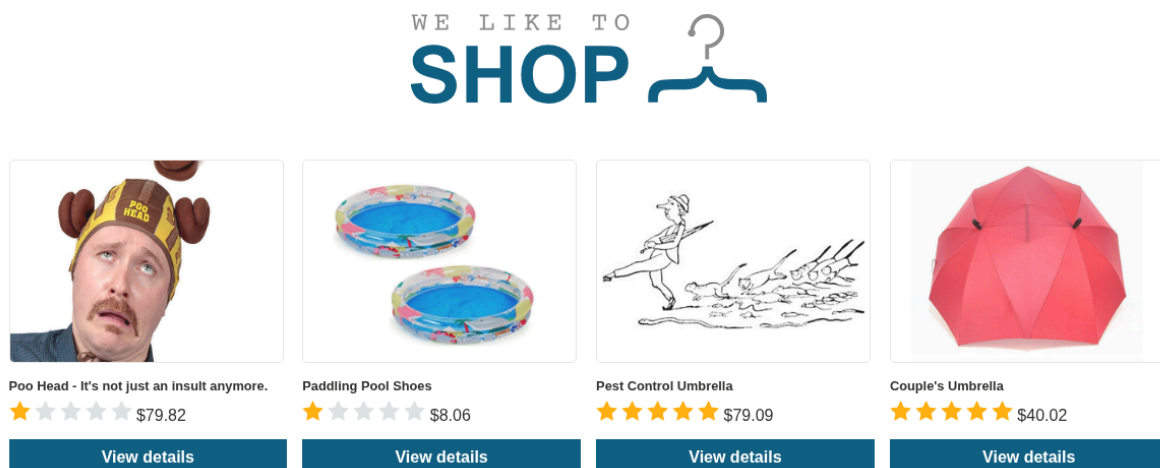
[Home](#)



En Burp Suite se desactiva temporalmente el interceptor ("Intercept is off") para permitir la carga completa de la página de la tienda.

Se navega a la página de un producto (clic en "View details"), lo que dispara la petición que carga la imagen del producto.

[Home](#)



Se busca en el historial (pestaña Proxy > HTTP history) la petición GET que carga la imagen (ejemplo: GET /image?filename=15.jpg) y se selecciona con clic derecho → "Send to Repeater".

Burp Suite Community Edition v2026.3.2 - Temporary Project
 Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Lc
 Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Request to https://0a870073047494658

Time	Type	Direction	Method	URL
23:23:49 ...	WS	→ To server		https://0a87007304749465801a1c23006a00cf.web-security-academy.net/academyLabHea
23:24:03 ...	HTTP	→ Request	GET	https://0a87007304749465801a1c23006a00cf.web-security-academy.net/image?filename=
23:24:03 ...	HTTP	→ Request	GET	https://0a87007304749465801a1c23006a00cf.web-security-academy.net/image?filename=42.jpg

Add to scope
 Forward
 Drop
 Add notes
 Highlight
 Don't intercept requests
 Do intercept
 Scan
 Send to Intruder Ctrl+I
 Send to Repeater Ctrl+R
 Send to Sequencer
 Send to Organizer Ctrl+O
 Send to Comparer
 Request in browser

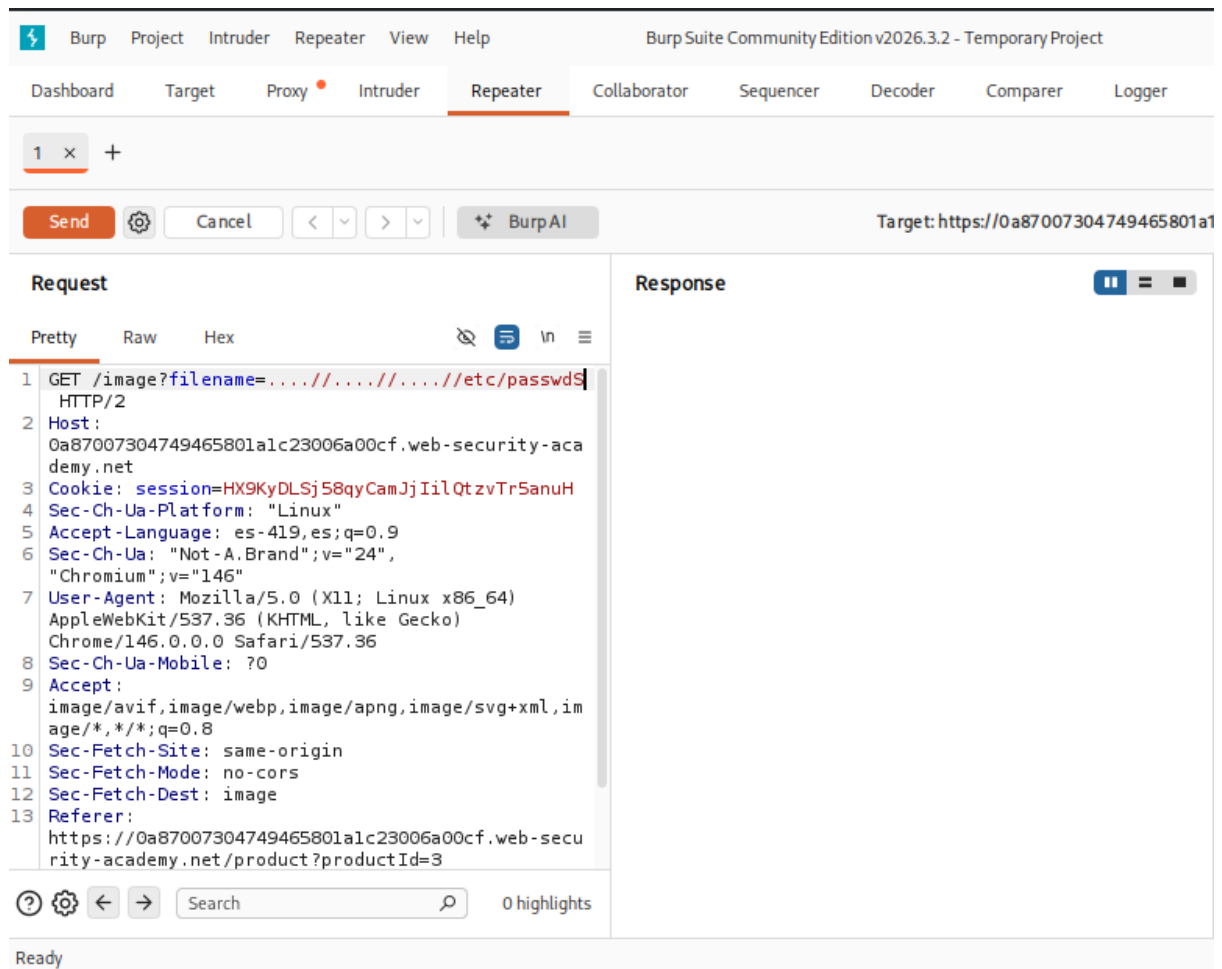
Request
 Pretty Raw Hex

```

1 GET /image?filename=42.jpg HTTP/2
2 Host: 0a87007304749465801a1c23006a00cf.web-security-academy.net
3 Cookie: session=HX9KyDLSj58qyCamJjIil0
4 Sec-Ch-Ua-Platform: "Linux"
5 Accept-Language: es-419,es;q=0.9
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="110"
7 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.0.0 Safari/537.36
  
```

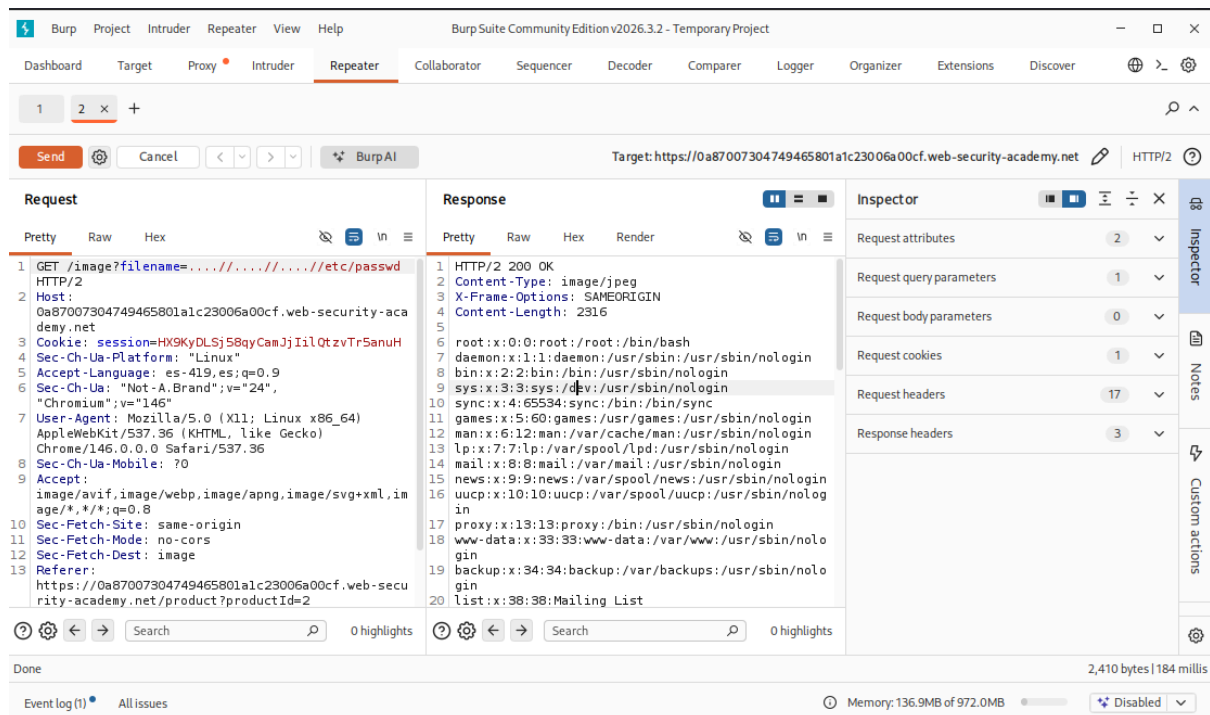
Search 0 h

En la línea 1 del request, dentro de la pestaña Repeater, se modifica el valor del parámetro filename, inyectando el payload ofuscado que evadirá el filtro:
 GET /image?filename=....//....//....//etc/passwd HTTP/2

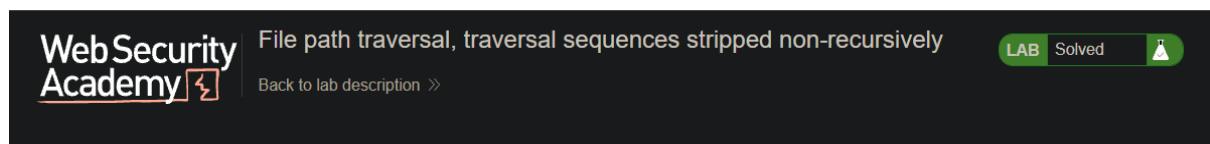


Se presiona "Send".

En el panel Response se observa una respuesta HTTP/2 200 OK. Al examinar el cuerpo de la respuesta, se visualizará el contenido completo del archivo /etc/passwd del servidor, validando que el filtro eliminó el contenido interior pero dejó pasar nuestra secuencia armada:



Esto confirma que la técnica de anidación evadió la medida de seguridad.
Se regresa a la pestaña del laboratorio en el navegador y se recarga la página (F5).
Apareció el banner verde: LAB Solved!



7. Explicación del payload

Payload utilizado://....//....//etc/passwd

-// : Es una secuencia de salto de directorio ofuscada mediante anidación.
- Estructura enviada: ..[../]/
- El filtro de la aplicación detecta y elimina estrictamente el patrón ../ que se encuentra en el interior de los corchetes ilustrativos.

- Una vez eliminado, los dos puntos iniciales (..) se concatenan con la barra diagonal final (/), dando como resultado una secuencia limpia y funcional: ../.
- Al concatenar esta ofuscación tres veces (....//....//....//), nos aseguramos de que, tras pasar por el filtro, el backend reciba ../.././, lo cual es suficiente para salir de los directorios web y llegar a la raíz del sistema de archivos (/).
- etc/passwd : La ruta del archivo objetivo que contiene la lista de usuarios del sistema Linux, demostrando la lectura no autorizada.

8. Mitigación recomendada

- Evitar listas negras y funciones de reemplazo: Nunca se deben utilizar funciones como `replace()` para intentar limpiar o sanitizar entradas maliciosas (listas negras). Los atacantes siempre encontrarán formas de evadirlas (anidación, codificación URL, byte nulo, etc.).
- Uso de identificadores indirectos: Como estándar de oro, no pasar el input del usuario a las APIs del sistema de archivos. Referenciar los archivos mediante un hash o un ID de base de datos (por ejemplo, ID=71 corresponde internamente a `/var/www/images/producto71.jpg`).
- Validación con ruta canónica (Canonicalización): Si se debe procesar el nombre del archivo enviado por el usuario, la aplicación debe resolver la ruta absoluta real (usando APIs como `getCanonicalPath()` o `realpath()`) que elimina cualquier secuencia matemática de directorios, y posteriormente verificar rigurosamente que esta ruta final comience con el directorio base permitido.
- Lista blanca (Whitelist) estricta: Validar que el valor del parámetro `filename` solo contenga caracteres alfanuméricos válidos y una extensión permitida (ej. solo letras, números y `.png/.jpg`), rechazando la solicitud por completo si contiene puntos o barras diagonales. No intentar "arreglar" la cadena, simplemente rechazarla.